COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***Information Security***
Instructor	***Mam Ambreen Gul***
Assignment No.	***03***

Student Details

Registration No.	***Name***
FA24-BSE-017	***Mubashir Hussain***

Task:01

Code:

```python
import hashlib   # Used to create SHA256 hashes
import time      # Used to get current timestamp
# ----------------------------
# Block class (represents a single block)
# ----------------------------
class Block:  2 usages
    def __init__(self, index, timestamp, data, previous_hash):
        self.index = index                  # Block number
        self.timestamp = timestamp          # Time when block is created
        self.data = data                    # Transaction data
        self.previous_hash = previous_hash  # Hash of previous block
        self.hash = self.calculate_hash()   # Hash of current block
    # Function to calculate SHA256 hash of the block
    def calculate_hash(self):  2 usages
        # Combine block data into a single string
        value = str(self.index) + str(self.timestamp) + str(self.data) + str(self.previous_hash)
        # Generate SHA256 hash
        return hashlib.sha256(value.encode()).hexdigest()
# ----------------------------
# Blockchain class (manages the chain of blocks)
# ----------------------------
class Blockchain:  1 usage
    def __init__(self):
        # Initialize blockchain with the genesis block
        self.chain = [self.create_genesis_block()]
    # Create the first block of the blockchain
    def create_genesis_block(self):  1 usage
        return Block( index: 0, time.ctime(),  data: "Genesis Block",  previous_hash: "0")
    # Get the most recent block in the chain
    def get_latest_block(self):  1 usage
        return self.chain[-1]
    # Add a new block to the blockchain
    def add_block(self, data):  4 usages
        previous_block = self.get_latest_block()   # Get last block
        new_block = Block(
            index=len(self.chain),               # New block index
            timestamp=time.ctime(),              # Current time
            data=data,                           # Transaction data
            previous_hash=previous_block.hash     # Link to previous block
        )
        self.chain.append(new_block)              # Add block to chain
    # Display all blocks in the blockchain
    def display_chain(self):  2 usages
        for block in self.chain:
            print("Index:", block.index)
            print("Timestamp:", block.timestamp)
            print("Data:", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Hash:", block.hash)
            print("-" * 50)
# ----------------------------
# Creating the Blockchain
# ----------------------------
my_blockchain = Blockchain()   # Create blockchain object
# Add 4 blocks with transaction data
my_blockchain.add_block("Transaction 1: A pays B")
my_blockchain.add_block("Transaction 2: B pays C")
my_blockchain.add_block("Transaction 3: C pays D")
my_blockchain.add_block("Transaction 4: D pays E")
# Display original blockchain
print("🔗 Original Blockchain:")
my_blockchain.display_chain()
# ----------------------------
# Tampering with a block
# ----------------------------
print("\n⚠️ Modifying Block 2 Data...\n")
# Change data of block at index 2
my_blockchain.chain[2].data = "Hacked Transaction"
# Recalculate hash after modification
my_blockchain.chain[2].hash = my_blockchain.chain[2].calculate_hash()
# Display blockchain after tampering
print("🔗 Blockchain After Tampering:")
my_blockchain.display_chain()
```

Code Explanation line by line:

import hashlib

Imports the hashlib library, which is used to generate SHA-256 cryptographic hashes.

import time

Imports the time module to generate timestamps for each block.

class Block:

Defines a Block class representing a single block in the blockchain.

__init__(self, index, timestamp, data, previous_hash)

Initializes a block with an index, timestamp, data, and the hash of the previous block.

self.hash = self.calculate_hash()

Automatically calculates the hash of the block to ensure data integrity.

calculate hash()

Combines all block attributes and generates a SHA-256 hash. Any change in block data results in a different hash.

class Blockchain:

Defines the Blockchain class that manages the list of blocks.

create_genesis_block()

Creates the first block of the blockchain, known as the Genesis Block.

add_block(data)

Adds a new block to the blockchain by linking it to the previous block using its hash.

display_chain()

Displays all blocks and their attributes, allowing us to observe the blockchain structure

Security Analysis

Blockchain security in this implementation is achieved through cryptographic hashing and block linking. SHA-256 hashing ensures data integrity, while linking blocks using previous hashes creates immutability. Any unauthorized modification is immediately detected. However, this implementation does not include encryption, consensus mechanisms, or peer-to-peer networking, which are present in real-world blockchains.

Output:

```
🔗 Original Blockchain:
Index: 0
Timestamp: Mon May  4 14:00:28 2026
Data: Genesis Block
Previous Hash: 0
Hash: ef898b94e7c6d6c5ce6c4912764a2a690e7e2c40542550543db55465eadd08bd
--------------------------------------------------
Index: 1
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 1: A pays B
Previous Hash: ef898b94e7c6d6c5ce6c4912764a2a690e7e2c40542550543db55465eadd08bd
Hash: 8aebed97e07ddce79f8bd0174ab7adcfe4f81e48bc08f37acb4613eeed369fd9
--------------------------------------------------
Index: 2
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 2: B pays C
Previous Hash: 8aebed97e07ddce79f8bd0174ab7adcfe4f81e48bc08f37acb4613eeed369fd9
Hash: e1a6ff3fb60a47cb1b7a5fdf26672a4bd329a19de59943c725a59966812351e4
--------------------------------------------------
Index: 3
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 3: C pays D
Previous Hash: e1a6ff3fb60a47cb1b7a5fdf26672a4bd329a19de59943c725a59966812351e4
Hash: f46e4b4ef83a8e15ed38c8f2780ff85ef9faaf296a515e4d76757ab0aae874dc
```

```
Index: 4
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 4: D pays E
Previous Hash: f46e4b4ef83a8e15ed38c8f2780ff85ef9faaf296a515e4d76757ab0aae874dc
Hash: b241291996ee8cf2e146e5a226e6d052bef1bc177819243452ba9550fe00a315
--------------------------------------------------

⚠️ Modifying Block 2 Data...

🔗 Blockchain After Tampering:
Index: 0
Timestamp: Mon May  4 14:00:28 2026
Data: Genesis Block
Previous Hash: 0
Hash: ef898b94e7c6d6c5ce6c4912764a2a690e7e2c40542550543db55465eadd08bd
--------------------------------------------------
Index: 1
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 1: A pays B
Previous Hash: ef898b94e7c6d6c5ce6c4912764a2a690e7e2c40542550543db55465eadd08bd
Hash: 8aebed97e07ddce79f8bd0174ab7adcfe4f81e48bc08f37acb4613eeed369fd9
--------------------------------------------------
Index: 2
Timestamp: Mon May  4 14:00:28 2026
Data: Hacked Transaction
Previous Hash: 8aebed97e07ddce79f8bd0174ab7adcfe4f81e48bc08f37acb4613eeed369fd9
Hash: a9d61f345b8416d1e8bb927a349fffc7e4b78f493c3f8da531c8be28747771f3
--------------------------------------------------
Index: 3
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 3: C pays D
Previous Hash: e1a6ff3fb60a47cb1b7a5fdf26672a4bd329a19de59943c725a59966812351e4
Hash: f46e4b4ef83a8e15ed38c8f2780ff85ef9faaf296a515e4d76757ab0aae874dc
--------------------------------------------------
Index: 4
Timestamp: Mon May  4 14:00:28 2026
Data: Transaction 4: D pays E
Previous Hash: f46e4b4ef83a8e15ed38c8f2780ff85ef9faaf296a515e4d76757ab0aae874dc
Hash: b241291996ee8cf2e146e5a226e6d052bef1bc177819243452ba9550fe00a315
--------------------------------------------------
```

Task:02

Code:

```python
# Import required libraries (no external crypto libraries used)
# ECC math is implemented manually for learning purpose
# Elliptic Curve parameters
a = 2                 # Curve parameter a
b = 3                 # Curve parameter b
p = 97                # Prime modulus
# ----------------------------------------
# Modular inverse function
# ----------------------------------------
def mod_inverse(k, p):  2 usages
    # Returns modular inverse of k modulo p
    return pow(k, -1, p)
# ----------------------------------------
# Point addition on elliptic curve
# ----------------------------------------
def point_add(P, Q):  1 usage
    # If P is point at infinity, return Q
    if P == "O":
        return Q
    # If Q is point at infinity, return P
    if Q == "O":
        return P
    # Extract x and y coordinates of points
    x1, y1 = P
    x2, y2 = Q
    # If x coordinates same but y different, result is infinity
    if x1 == x2 and y1 != y2:
        return "O"
    # If points are equal, perform point doubling
```

```
    if P == Q:
        return point_double(P)
    # Calculate slope for point addition
    m = ((y2 - y1) * mod_inverse(x2 - x1, p)) % p
    # Calculate x coordinate of result
    x3 = (m*m - x1 - x2) % p
    # Calculate y coordinate of result
    y3 = (m*(x1 - x3) - y1) % p
    # Return resulting point
    return (x3, y3)
# ----------------------------------------
# Point doubling
# ----------------------------------------
def point_double(P):  1 usage
    # If P is infinity, return infinity
    if P == "O":
        return "O"
    # Extract x and y coordinates
    x, y = P
    # Calculate slope for point doubling
    m = ((3*x*x + a) * mod_inverse(2*y, p)) % p
    # Calculate new x coordinate
    x3 = (m*m - 2*x) % p
    # Calculate new y coordinate
    y3 = (m*(x - x3) - y) % p
    # Return doubled point
    return (x3, y3)
```

```
def scalar_multiply(k, P):  4 usages
    # Initialize result as point at infinity
    result = "O"
    # Repeatedly add point P, k times
    for _ in range(k):
        result = point_add(result, P)
    # Return final multiplied point
    return result
# ----------------------------------------
# Generator point
# ----------------------------------------
G = (3, 6)             # Base point on the curve
# ----------------------------------------
# Key generation
# ----------------------------------------
private_key = 5       # Manually chosen private key
public_key = scalar_multiply(private_key, G)  # Public key
# Display keys
print("Private Key:", private_key)
print("Public Key:", public_key)
# ----------------------------------------
# Message
# ----------------------------------------
message = 7           # Small numeric message
print("\nOriginal Message:", message)
def get_x(point):  2 usages
    if point == "O":
        raise ValueError("Shared secret is point at infinity. Choose different k or keys.")
    return point[0]
```

```
# Encryption
k = 3
C1 = scalar_multiply(k, G)
shared_secret = scalar_multiply(k, public_key)
C2 = message + get_x(shared_secret)
print("\nEncrypted Message:")
print("C1:", C1)
print("C2:", C2)
# Decryption
shared_secret_dec = scalar_multiply(private_key, C1)
decrypted_message = C2 - get_x(shared_secret_dec)
print("\nDecrypted Message:", decrypted_message)         # Recover message
# Display decrypted message
print("\nDecrypted Message:", decrypted_message)
```

Code Explanation line by line:

1. Overview:

The program demonstrates ECC key generation, encryption, and decryption using elliptic curve arithmetic over a finite field.

2. Curve Parameters:

- a = 2, b = 3 define the elliptic curve equation y^2 = x^3 + ax + b mod p - p = 97 defines the finite field

3. Modular Inverse:

Used to perform division in modular arithmetic using pow(k, -1, p).

4. Point Operations:

- Point addition handles combining two points on the curve
- Point doubling handles adding a point to itself- Special case: "O" represents point at infinity

5. Scalar Multiplication:

Repeated point addition used to compute kP.

6. Key Generation:

- Private key is a secret integer
- Public key is computed as private_key * G

7. Encryption:

- Uses ephemeral key k
- Computes C1 = kG
- Shared secret = k * public_key
- C2 = message + x-coordinate(shared secret)

8. Decryption:

- Recomputes shared secret using private key- Recovers message by subtracting x-coordinate SECURITY ANALYSIS:

Strengths:

- Correct ECC mathematical structure
- Based on Elliptic Curve Discrete Logarithm Problem- Demonstrates secure key exchange concept

Weaknesses:

- Very small prime field (p = 97) makes brute force easy
- Scalar multiplication is inefficient (O(k))
- Ephemeral key k is not random
- No hashing or key derivation function
- No authentication or integrity protection
- Simplified encryption (message + x-coordinate)

Output:

```
D:\PythonProject\PythonProject\PythonProject\ISpractice\.venv\Scripts\python.exe D:\PythonProject\PythonProject\PythonProject\ISpractice\ISL
Private Key: 5
Public Key: 0

Original Message: 7
```